

DIPARTIMENTO DI INFORMATICA

Corso di Laurea in Informatica

Tesi di Laurea

Studio e verifica dei flussi IEC 61850 tramite libIEC61850 e analisi Wireshark

Relatore universitario:

Prof. Elvio Gilberto Amparore

Relatore aziendale:

Dott. Federico Cerruti

Sede del tirocinio:

Col Group – Trofarello

Candidato:

Mat. 867140

Plaia Antonino

Periodo di svolgimento: 28/09/2026 – 05/11/2026

ANNO ACCADEMICO 2025/2026

Indice

1	Introduzione	1
1.1	Lo standard IEC 61850	1
1.1.1	Modello dati virtualizzato	1
1.1.2	Servizi di comunicazione	2
1.1.3	Il linguaggio di configurazione SCL	3
1.1.4	Architettura della sottostazione	3
1.1.5	Le parti dello standard	3
1.2	Il tirocinio	3
2	Obiettivo	4
2.1	Finalità del progetto	4
2.2	Architettura proposta	4
2.3	Componenti software	5
2.3.1	Orchestratore dei test	5
2.3.2	Client IEC 61850	5
2.3.3	Acquisizione e validazione con Wireshark e tshark	5
2.3.4	Interfaccia web (componente opzionale)	6
2.4	Scenari di test	6
2.5	Elaborati previsti	6
3	Contesto lavorativo	8
3.1	L'azienda	8
3.2	L'unità BU9020	8
3.3	La piattaforma di simulazione dei segnali	8
3.4	Ambiente di lavoro	8
3.5	Background tecnico	8
3.5.1	Modello dati e riferimenti a oggetto	9
3.5.2	Logical Node 0 e nodi funzionali	9
3.5.3	Servizi di report	9
3.5.4	GOOSE	10
4	Processo lavorativo	11
4.1	Metodologia	11
4.2	Studio dello standard e preparazione del banco	11

4.3	Sviluppo dell'orchestratore e del protocollo seriale	11
4.4	Sviluppo del client libIEC61850	11
4.5	Cattura e analisi del traffico	11
4.6	Esecuzione degli scenari	12
5	Risultati ottenuti	13
5.1	Esiti degli scenari	13
5.2	Tempi misurati	13
5.3	Anomalie riscontrate	13
6	Bilancio dell'esperienza formativa	14
6.1	Competenze acquisite	14
6.2	Difficoltà incontrate	14
6.3	Collegamento con il percorso di studi	14
7	Conclusioni	15
	Bibliografia	16

Capitolo 1

Introduzione

Una sottostazione elettrica è il nodo della rete di trasmissione e distribuzione in cui la tensione viene trasformata, l'energia smistata e i guasti isolati. Le apparecchiature che la compongono, come interruttori, sezionatori, trasformatori di misura e relè di protezione, sono controllate e monitorate da dispositivi elettronici intelligenti, detti IED (*Intelligent Electronic Device*), che scambiano informazioni tra loro e con il sistema di supervisione della stazione.

Storicamente questo scambio è avvenuto tramite protocolli diversi, spesso specifici del costruttore, e attraverso cablaggi in rame dedicati a ogni singolo segnale. L'integrazione di apparati di produttori differenti richiedeva quindi dispositivi di conversione di protocollo (*gateway*), che introducono ritardi e possibili errori nello scambio dei dati e aumentano i costi di progettazione, messa in servizio e manutenzione.

1.1 Lo standard IEC 61850

Per superare questi limiti la Commissione Elettrotecnica Internazionale (IEC) ha sviluppato lo standard IEC 61850, *Communication networks and systems for power utility automation* [1]. Il suo obiettivo è l'interoperabilità: dispositivi di costruttori diversi devono poter scambiare informazioni senza conversioni, indipendentemente dalle caratteristiche fisiche di ciascun apparato. **[DA COMPLETARE: edizione dello standard supportata dal dispositivo in esame]**

Lo standard si fonda su tre idee principali: un modello dati virtualizzato, la separazione tra servizi di comunicazione e protocolli concreti, un linguaggio formale di configurazione.

1.1.1 Modello dati virtualizzato

Ogni dispositivo viene descritto in modo astratto, in base alle funzioni che svolge e alle informazioni che espone, e non in base al suo hardware. Un dispositivo fisico (IED) contiene uno o più *Logical Device*, ciascuno composto da *Logical Node* che rappresentano singole funzioni. Ad esempio XCBR modella un interruttore, MMXU le misure, RSYN il controllo di sincronismo, RREC la richiusura automatica e ATCC la regolazione automatica del variatore di prese del trasformatore. **[DA COMPLETARE: verificare nel file ICD quali nodi logici espone effettivamente l'unità in esame]**

A loro volta i nodi logici contengono *Data Object*, a cui sono associati *Data Attribute*, con nomi e significati uniformi per tutti i costruttori. Un riferimento a un attributo ha quindi la forma

`LogicalDevice/LogicalNode.DataObject.DataAttribute`. I dati che devono essere trasmessi insieme vengono raggruppati in un *data set*.

1.1.2 Servizi di comunicazione

I servizi di scambio informativo sono definiti in forma astratta (ACSI, *Abstract Communication Service Interface*) e poi associati a protocolli concreti. I principali sono:

- MMS (*Manufacturing Message Specification*) su TCP/IP, per la comunicazione client-server;
- GOOSE (*Generic Object Oriented Substation Event*), per messaggi rapidi in multicast direttamente su Ethernet, usati per interblocchi e segnali di sgancio tra protezioni;
- *Sampled Values* (SV), per la trasmissione dei campioni digitalizzati di correnti e tensioni.

Poiché il presente lavoro riguarda MMS e GOOSE, i due servizi sono descritti più in dettaglio.

MMS. Il servizio è basato sul modello client-server ed è trasportato su TCP/IP (tipicamente sulla porta 102). Un client, ad esempio un sistema di supervisione, può leggere e scrivere i valori dei dati dell'IED, inviare comandi e ricevere rapporti (*report*) spontanei configurati tramite *Report Control Block*, bufferizzati o non bufferizzati, che inviano i dati al client al verificarsi di determinate condizioni.

GOOSE. Il servizio segue il modello *publisher-subscriber*: un IED pubblica il contenuto di un data set e gli altri IED interessati lo ricevono. I messaggi viaggiano direttamente su Ethernet (livello 2), senza passare da IP e TCP, in multicast e con un EtherType dedicato, per ridurre la latenza. In caso di cambio di stato il messaggio viene inviato subito e poi ritrasmesso a intervalli inizialmente brevi e progressivamente crescenti, fino a un intervallo massimo di regime, così da recuperare eventuali perdite senza attendere conferme. I campi del messaggio rilevanti per la verifica sono:

- `gocbRef`: riferimento al *GOOSE Control Block* che pubblica il messaggio;
- `datSet`: riferimento al data set pubblicato;
- `goID`: identificatore del messaggio GOOSE;
- `timeAllowedToLive`: tempo massimo che il ricevitore attende prima di considerare perso il messaggio successivo;
- `t`: marca temporale dell'ultimo cambio di stato;
- `stNum`: numero di stato, incrementato a ogni cambio di valore del data set;
- `sqNum`: numero di sequenza, incrementato a ogni ritrasmissione con lo stesso `stNum` e riportato a zero quando `stNum` cambia;
- il contenuto del data set (*payload* applicativo).

[DA COMPLETARE: confrontare la descrizione di `stNum/sqNum` e delle ritrasmissioni con la parte 8-1 dello standard e con il materiale fornito]

1.1.3 Il linguaggio di configurazione SCL

Il linguaggio SCL (*Substation Configuration Language*), basato su XML, permette di descrivere in modo formale i dispositivi, la sottostazione e la rete di comunicazione, e di scambiare le configurazioni tra strumenti di costruttori diversi. I file di configurazione hanno estensioni diverse a seconda dello scopo: ICD (capacità di un singolo IED), CID (configurazione di un IED), SCD (descrizione dell'intero sistema).

1.1.4 Architettura della sottostazione

L'architettura di una sottostazione IEC 61850 è organizzata su tre livelli: livello di stazione, livello di campo (*bay*, in italiano stallo) e livello di processo, collegati da reti Ethernet dette bus di stazione e bus di processo. Le funzioni di controllo di stallo, come quelle dell'unità considerata in questa tesi, si collocano al livello di campo.

1.1.5 Le parti dello standard

Lo standard è suddiviso in più parti. Quelle rilevanti per il lavoro sono riassunte nella Tabella 1.1.

Parte	Contenuto
6	Linguaggio di configurazione SCL
7-2	Servizi astratti ACSI e modello informativo di base
7-3	Classi di dati comuni
7-4	Classi di nodi logici compatibili e oggetti dato
8-1	Associazione dei servizi a MMS ed Ethernet (client-server e GOOSE)
9-2	Associazione dei <i>Sampled Values</i> a Ethernet
10	Test di conformità

Tabella 1.1: Parti dello standard IEC 61850 rilevanti per il lavoro svolto.

Il presente lavoro non è una procedura di certificazione secondo la parte 10, ma una verifica funzionale del comportamento del dispositivo in scenari controllati. **[DA COMPLETARE: confermare questa impostazione con il relatore aziendale]**

1.2 Il tirocinio

Il presente lavoro nasce dal tirocinio curricolare svolto presso Col Group, a Trofarello, nel periodo dal 28 settembre al 5 novembre 2026.

L'obiettivo del tirocinio è lo studio e la verifica dei flussi di comunicazione IEC 61850 generati e ricevuti da un'unità di stallo, la BU9020, mediante un ambiente di test su Linux. Per la realizzazione si utilizzano libIEC61850 [2], libreria open source scritta in C che implementa i servizi MMS, GOOSE e SV, e Wireshark [3] insieme al suo strumento a riga di comando **tshark**, per la cattura e l'analisi del traffico di rete.

Le attività svolte sono state **[DA COMPLETARE: riassumere in 3-5 righe cosa è stato fatto]**. I risultati principali sono **[DA COMPLETARE: da scrivere a lavoro concluso, con dati concreti]**.

Capitolo 2

Obiettivo

2.1 Finalità del progetto

L'obiettivo è progettare e realizzare un ambiente di test per gestire una scheda di segnali analogici e digitali e analizzare i flussi IEC 61850 generati o ricevuti dalla BU9020 durante scenari funzionali controllati.

Il lavoro correla tre livelli:

1. **Stimolo fisico simulato.** Comandi inviati via seriale a una piattaforma basata su ESP32, che generano condizioni analogiche e digitali in ingresso all'unità.
2. **Comportamento applicativo della BU9020.** Attivazione di funzioni come sincronismo, richiusura o regolazione di tensione in risposta agli stimoli.
3. **Traffico IEC 61850 sulla LAN.** Messaggi MMS e GOOSE osservati tramite un client basato su libIEC61850 e tramite cattura Wireshark.

Il risultato atteso non è una semplice cattura passiva del traffico, ma una procedura ripetibile in grado di stabilire se un determinato scenario produce il flusso IEC 61850 atteso. Il simulatore di segnali è già stato realizzato dall'azienda e non fa parte del contributo del lavoro: questo consiste nello studio del protocollo e nella costruzione di un *framework* che automatizzi il ciclo di verifica.

2.2 Architettura proposta

L'architettura, illustrata in Figura 2.1, prevede un computer a scheda singola (SBC) ARM con sistema operativo Linux che svolge il ruolo di orchestratore. L'SBC è collegato in due modi: via seriale (USB/UART) alla piattaforma ESP32, che genera i segnali analogici e digitali applicati alla BU9020, e via Ethernet alla stessa LAN della BU9020, per interrogarla con il client IEC 61850 e catturare il traffico.

Il ciclo di verifica che il sistema deve automatizzare è il seguente: lo scenario di test genera i comandi, l'orchestratore li invia via seriale al simulatore, i segnali fisici raggiungono la BU9020, questa produce traffico IEC 61850 sulla rete, il traffico viene catturato e analizzato, e l'esito è riportato in un rapporto di superamento o fallimento (*pass/fail*).

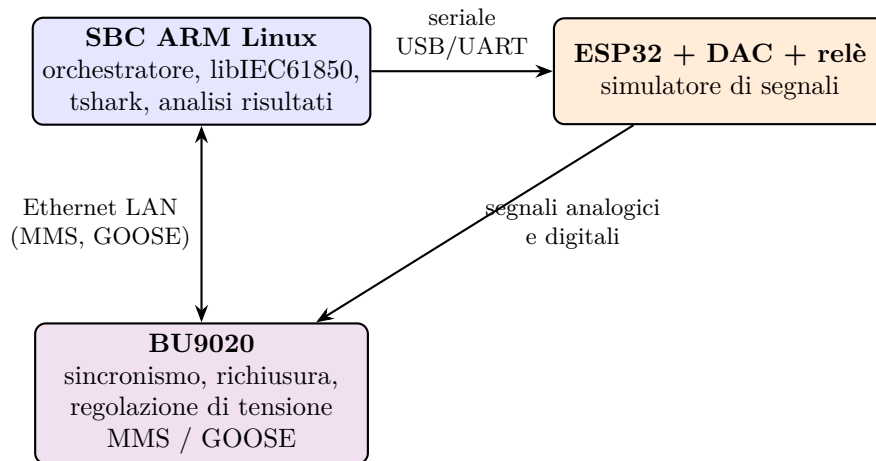


Figura 2.1: Architettura del banco di prova.

2.3 Componenti software

2.3.1 Orchestratore dei test

L'orchestratore è un programma, o un insieme di script, preferibilmente in Python, che si occupa di: caricare la descrizione di uno scenario di test; inviare i comandi seriali alla piattaforma ESP32; avviare la cattura del traffico di rete; interrogare la BU9020 tramite il client IEC 61850; raccogliere risultati e registri (*log*); generare un rapporto di esito.

2.3.2 Client IEC 61850

Il client, basato su libIEC61850, deve consentire almeno la connessione MMS alla BU9020, la lettura di oggetti dato, la verifica di attributi IEC 61850 e la correlazione dei valori letti con gli eventi generati. Sono inoltre previsti, se supportati dal setup, l'attivazione o il controllo di un *Report Control Block* e la sottoscrizione o l'analisi dei messaggi GOOSE.

Un esempio di verifica è il seguente. All'evento simulato «perdita di sincronismo tra tensione di linea e di sbarra» deve corrispondere l'aggiornamento dello stato logico interno, l'eventuale blocco del comando, la pubblicazione di un messaggio GOOSE con stato «non sincronizzato» e la lettura via MMS di valori coerenti. La parte più interessante del lavoro è trasformare il client da semplice strumento di lettura a componente di verifica automatica.

2.3.3 Acquisizione e validazione con Wireshark e tshark

Wireshark rimane lo strumento di riferimento per la validazione manuale e diagnostica, mentre *tshark* permette di estrarre automaticamente i campi dai file di cattura *.pcapng*. Le verifiche previste riguardano:

- la presenza dei frame GOOSE attesi;
- la correttezza dei campi *gocbRef*, *datSet*, *goID*, *stNum*, *sqNum*, *timeAllowedToLive*, della marca temporale e del contenuto applicativo;
- la sequenza dei messaggi dopo un evento e le ritrasmissioni dopo un cambio di stato;

- la coerenza tra cambio di stato e incremento di `stNum`;
- la latenza tra l'evento e il frame GOOSE corrispondente;
- l'assenza di frame inattesi.

2.3.4 Interfaccia web (componente opzionale)

Come obiettivo secondario, e solo se il tempo lo consente, l'orchestratore può esporre un'interfaccia web HTTP/REST in esecuzione sull'SBC. Il server web non comunica direttamente con l'ESP32: ogni comando passa attraverso l'orchestratore, che lo inoltra via seriale al simulatore. Le funzioni previste sono il controllo remoto del simulatore, un pannello di monitoraggio dello stato dei segnali e della connessione con la BU9020, la gestione degli scenari di test e un'API per l'automazione, che permette anche l'integrazione con sistemi di integrazione continua. **[DA COMPLETARE: indicare se questa componente è stata realizzata]**

2.4 Scenari di test

Sono previsti tre scenari, corrispondenti alle funzioni della BU9020 riassunte nella Tabella 2.1.

Scenario	Grandezze controllate	Verifiche IEC 61850
Controllo di sincronismo	differenza di fase e di tensione; differenza di frequenza, se supportata dalla piattaforma; stato digitale di abilitazione del comando	stato del consenso al sincronismo; eventuale GOOSE; lettura MMS dello stato associato; latenza tra stimolo e aggiornamento
Richiusura automatica	apertura dell'interruttore; simulazione di guasto; rientro delle condizioni corrette; abilitazione o inibizione della richiusura	sequenza degli stati; comandi e feedback associati; messaggi GOOSE generati; corretto incremento di <code>stNum</code> ; tempi tra gli eventi
Regolazione di tensione del trasformatore	tensione misurata; soglie di regolazione; ingressi digitali di blocco o consenso; eventuali feedback di posizione	stato del regolatore; richieste di salita e discesa prese; messaggi GOOSE relativi; valori MMS esposti; coerenza tra scenario e comando atteso

Tabella 2.1: Scenari di test previsti.

[DA COMPLETARE: indicare quali scenari sono stati effettivamente implementati e testati]

2.5 Elaborati previsti

Il lavoro prevede di produrre:

1. un ambiente Linux di test (configurazione dell'SBC, gestione della seriale verso l'ESP32, gestione della cattura di rete);

2. un client IEC 61850 in C basato su libIEC61850, integrato nel flusso automatico di test;
3. una suite di test in Python, con scenari descritti in YAML o JSON, stimoli, condizioni attese e criteri di superamento;
4. un analizzatore dei pacchetti basato su `tshark`, che estrae i campi IEC 61850 e verifica automaticamente i frame GOOSE;
5. un rapporto finale con gli esiti, i tempi misurati, i messaggi osservati e le eventuali anomalie.

[DA COMPLETARE: aggiornare l'elenco con quanto realizzato]

Capitolo 3

Contesto lavorativo

3.1 L'azienda

[DA COMPLETARE: descrivere Col Group: attività, settore, dimensioni, sede di Trofarello, e il reparto in cui si è svolto il tirocinio]

3.2 L'unità BU9020

La BU9020 è un'unità destinata ad applicazioni di stallo in ambito elettrico. Fornisce funzioni di controllo e monitoraggio, tra cui: controllo di sincronismo, richiusura automatica, regolazione della tensione del trasformatore, gestione di ingressi e uscite digitali, pubblicazione e ricezione di messaggi IEC 61850. [DA COMPLETARE: aggiungere una descrizione più completa dell'unità: ruolo nel prodotto aziendale, interfacce, versione del firmware]

3.3 La piattaforma di simulazione dei segnali

Per stimolare la BU9020 l'azienda ha già realizzato una piattaforma di prova basata su un microcontrollore ESP32, un convertitore digitale-analogico (DAC) e dei relè. La piattaforma è in grado di generare segnali analogici trifase e stati digitali controllati e può essere pilotata tramite interfaccia UART. [DA COMPLETARE: specificare i comandi seriali disponibili, le grandezze e i limiti di generazione (ad esempio la frequenza)]

3.4 Ambiente di lavoro

[DA COMPLETARE: descrivere l'SBC ARM utilizzato (modello, sistema operativo), la rete di test, gli strumenti e le modalità di lavoro in azienda]

3.5 Background tecnico

Questa sezione riprende dallo standard IEC 61850 gli elementi necessari per interpretare il traffico catturato e il codice del client. Il riferimento sono le parti 7-2 (servizi astratti), 7-4 (nodi logici) e 8-1 (associazione a MMS) [indicare titolo, anno ed edizione di ciascuna parte].

3.5.1 Modello dati e riferimenti a oggetto

Un dispositivo (*server*) contiene uno o più *Logical Device*, ciascuno composto da *Logical Node*, che contengono *Data Object* e *Data Attribute*. Ogni oggetto ha un nome, univoco solo nel proprio livello, e un riferimento, univoco nell'intero sistema, ottenuto concatenando i nomi lungo il percorso. Ogni attributo è inoltre associato a un *Functional Constraint* (FC), che indica a quale funzione appartiene: ad esempio ST per lo stato, MX per le misure, CO per i comandi.

Livello	Nome	Riferimento
Logical Device	LD1	LD1
Logical Node	XCBR1	LD1/XCBR1
Data Object	Pos	LD1/XCBR1.Pos
Data Attribute	stVal	LD1/XCBR1.Pos.stVal

Tabella 3.1: Gerarchia del modello dati per la posizione di un interruttore.

Nel mapping su MMS il server corrisponde a un *VMD*, ogni Logical Device a un *domain* e ogni dato a una *named variable*. Il riferimento viene quindi diviso nel punto della barra, i punti sono sostituiti dal carattere \$ e il Functional Constraint è inserito dopo il nome del nodo:

IEC 61850:	LD1/XCBR1.Pos.stVal	FC = ST
MMS:	domainId = "LD1"	
	itemId = "XCBR1\$ST\$Pos\$stVal"	

Queste due stringhe sono quelle che compaiono nelle richieste di lettura analizzate con Wireshark (Capitolo 5). [\[Verificare se nella BU9020 il domainId include il nome dello IED.\]](#)

3.5.2 Logical Node 0 e nodi funzionali

Ogni Logical Device contiene obbligatoriamente il nodo LLN0, che rappresenta il dispositivo logico nel suo insieme, e il nodo LPHD, che rappresenta il dispositivo fisico, oltre ad almeno un nodo funzionale (ad esempio XCBR per un interruttore o MMXU per le misure). La differenza pratica è che i *control block* dei servizi multicast (GOOSE e Sampled Values) e dei gruppi di parametri sono definiti solo in LLN0, mentre i nodi funzionali possono contenere solo i control block dei report e dei log. Per questo i riferimenti ai control block hanno forma LD1/LLN0\$RP\$. . . o LD1/LLN0\$GO\$. . .

3.5.3 Servizi di report

Il *report* è il servizio con cui il server invia al client, in modo spontaneo, i valori di un *data set*, cioè di un elenco ordinato di riferimenti a dati. L'invio è governato da un *Report Control Block*, che può essere *buffered* (gli eventi sono conservati se il client è disconnesso) oppure *unbuffered* (gli eventi in assenza del client sono persi). Il client abilita il report scrivendo l'attributo RptEna.

Il motivo per cui un report viene inviato è espresso dalle *trigger condition*, riassunte in Tabella 3.2. Un invio avviene solo se la condizione è abilitata nel control block e l'attributo la supporta. Ogni dato incluso in un report riporta la causa di inclusione, ed è il campo con cui, nell'analisi, si distinguono i report generati da eventi da quelli periodici o richiesti dal client.

Condizione	Quando si verifica
dchg	cambia il valore di un dato
qchg	cambia la qualità di un dato
dupd	il dato viene aggiornato, anche senza variare
integrity	scade il periodo <code>IntgPd</code> : vengono inviati tutti i valori
GI	il client richiede la lettura completa del data set

Tabella 3.2: Condizioni che possono provocare l’invio di un report.

3.5.4 GOOSE

I messaggi GOOSE sono inviati in multicast direttamente su Ethernet (Ethertype `0x88B8`), senza passare da IP e TCP, con un meccanismo di tipo *publisher-subscriber* privo di conferme. L’affidabilità è ottenuta ripetendo il messaggio: dopo una variazione dei dati il publisher lo ritrasmette a intervalli crescenti fino a un intervallo di regime. Per la verifica dei flussi sono significativi i campi in Tabella 3.3.

Campo	Significato
stNum	numero di stato: cresce a ogni variazione dei dati
sqNum	numero di sequenza: riparte da zero a ogni variazione e cresce a ogni ripetizione
confRev	revisione della configurazione del data set
timeAllowedToLive	tempo oltre il quale il subscriber considera perso il publisher

Tabella 3.3: Campi del messaggio GOOSE usati nell’analisi.

Un salto di **sqNum** indica quindi un messaggio perso, mentre una variazione di **stNum** indica un evento.

Capitolo 4

Processo lavorativo

4.1 Metodologia

[**DA COMPLETARE:** descrivere l'organizzazione del lavoro: fasi, incontri con il relatore aziendale, strumenti di versionamento e documentazione]

4.2 Studio dello standard e preparazione del banco

[**DA COMPLETARE:** materiale studiato, configurazione del banco, versioni di libIEC61850 e di Wireshark utilizzate]

4.3 Sviluppo dell'orchestratore e del protocollo seriale

[**DA COMPLETARE:** struttura del programma, formato degli scenari (YAML/JSON), comandi inviati all'ESP32]

4.4 Sviluppo del client libIEC61850

[**DA COMPLETARE:** funzionalità implementate, come è stata gestita la connessione MMS e, se presente, la sottoscrizione GOOSE]

4.5 Cattura e analisi del traffico

L'estrazione dei campi GOOSE da un file di cattura avviene con **tshark**. Un esempio di comando è il seguente:

```
tshark -r test_sync_check_ok.pcapng \  
-Y "goose" \  
-T fields \  
-e frame.time_epoch \  
-e eth.src \  
-e goose.gocbRef \  
-e goose.datSet \  
-e goose.goID \  

```

```
-e goose.stNum \  
-e goose.sqNum
```

[**DA COMPLETARE:** filtri e comandi effettivamente usati, come vengono automatizzate le verifiche]

4.6 Esecuzione degli scenari

[**DA COMPLETARE:** procedura di esecuzione di uno scenario e criteri di pass/fail adottati]

Capitolo 5

Risultati ottenuti

5.1 Esiti degli scenari

[**DA COMPLETARE:** per ogni scenario: esito, messaggi osservati, confronto con il comportamento atteso]

5.2 Tempi misurati

[**DA COMPLETARE:** latenze tra stimolo, evento e frame GOOSE, con il metodo di misura e la sua incertezza]

5.3 Anomalie riscontrate

[**DA COMPLETARE:** comportamenti inattesi della BU9020 o del banco di prova e loro analisi]

Capitolo 6

Bilancio dell'esperienza formativa

6.1 Competenze acquisite

[DA COMPLETARE: competenze tecniche e trasversali]

6.2 Difficoltà incontrate

[DA COMPLETARE: difficoltà tecniche e organizzative e come sono state risolte]

6.3 Collegamento con il percorso di studi

[DA COMPLETARE: insegnamenti che sono risultati utili e conoscenze che mancavano]

Capitolo 7

Conclusioni

[**DA COMPLETARE:** efficacia dell'ambiente di test, limiti del lavoro, sviluppi futuri, ad esempio l'integrazione in una pipeline di test più ampia]

Bibliografia

- [1] IEC, *IEC 61850: Communication networks and systems for power utility automation*, serie di norme. **[DA COMPLETARE: indicare le parti e le edizioni consultate]**
- [2] MZ Automation GmbH, *libIEC61850*, libreria open source. <https://libiec61850.com>. **[DA COMPLETARE: data di consultazione e versione]**
- [3] Wireshark Foundation, *Wireshark*. <https://www.wireshark.org>. **[DA COMPLETARE: data di consultazione e versione]**
- [4] CESI, *FIA Informatica e Automazione*, 2005. **[DA COMPLETARE: completare i dati del documento e citarlo nel testo dove è stato usato]**